UNIVERSIDADE FEDERAL
DE SANTA CATARINA
Centro de Ciências, Tecnologias
e Saúde - CTS

Nome do Aluno: Diego Nyland Bloemer (24103789); Davi Ferreira da Silveira (24102025).
Professora: Andréa Sabedra Bordin.
Disciplina: DEC7130 - Engenharia de Software II.

TRABALHO PRÁTICO DA UNIDADE II

Araranguá, SC
2026

SUMÁRIO

1 IDENTIFICAÇÃO DO PROJETO ORIGINAL 3
2 CARACTERIZAÇÃO E COMPREENSÃO INICIAL DO SISTEMA 3
2.1 BASELINE 4
3 PROBLEMAS E REFATORAÇÃO 5
3.1 FALHAS DE MANUTENIBILIDADE IDENTIFICADOS 5
3.1.1 Método datetime.utcnow() Descontinuado 5
3.1.2 Navbar não é Atualizada após o Registro/Login 6
3.1.3 Dados Enviados Incorretamente para API 6
3.2 PROBLEMAS DE MANUTENIBILIDADE IDENTIFICADOS 7
3.2.1 Método e Dependências Instanciadas e Não Utilizadas 7
3.2.2 God Component / Mistura de Responsabilidades no Front-end 9
3.2.3 Fat Route / Regra de Negócio e Persistência Misturadas na Rota 12
3.3 REFATORAÇÃO REALIZADA 14
4 ANÁLISE, SOLICITAÇÃO E IMPLEMENTAÇÃO DE MANUTENÇÃO 16
4.1 SOLICITAÇÃO DE MANUTENÇÃO 16
4.2 ANÁLISE DE IMPACTO 17
4.3 DESCRIÇÃO DA IMPLEMENTAÇÃO 18
4.3.1 BACK-END 18
4.3.2 FRONT-END 19
4.4 COMPARAÇÃO ENTRE IMPACTO PREVISTO E OBSERVADO 20
5 REFLEXÃO E CONCLUSÃO 21

1 IDENTIFICAÇÃO DO PROJETO ORIGINAL

Elemento	Descrição
Descrição do Projeto	Uma plataforma de coordenação de doação de alimentos que conecta supermercados, restaurantes e ONGs para distribuir o excedente de alimentos.
Link do Repositório	https://github.com/Giftmbatha/food-rescue
Justificativa da escolha	● O projeto está alinhado com as demais atividades e projetos já desenvolvidas nas disciplinas de Engenharia de Software I e Engenharia de Software II e visa dar seguimento agora com a refatoração e manutenção de um projeto que possui como objetivo unir doadores de produtos ou excedentes a ONGs interessadas; ● O projeto é desenvolvido em linguagens de maior domínio e facilidade para os dois integrantes do grupo;

2 CARACTERIZAÇÃO E COMPREENSÃO INICIAL DO SISTEMA

Elemento	Descrição
Qual o objetivo do software?	● Plataforma de coordenação de doações de alimentos. ● Conecta supermercados e restaurantes doadores a ONGs para distribuir alimentos excedentes e reduzir desperdícios.
Quais as linguagens e tecnologias utilizadas?	● Linguagens: Python, JavaScript. Tecnologias: ● Frontend: React, React Router, Tailwind CSS e DaisyUI. ● Backend: FastAPI, Uvicorn, SQLModel e Pydantic. ● Banco de dados: SQLite. ● Autenticação: JWT e bcrypt.
Como o projeto está organizado?	A aplicação roda de forma desacoplada tendo o backend e o frontend serviços separados. Backend: aplicação da API em FastAPI. ● app/: código principal do backend. ● app/routers/: rotas de autenticação, doadores, ONGs, listagens e solicitações. ● tests/: testes de integração.

	Frontend: aplicação web em React. ● src/: componentes, páginas, rotas, autenticação e comunicação com a API.
Onde se encontra o código principal?	Backend: backend/app/; ● Ponto de entrada da API: backend/app/main.py; Frontend: frontend/src/. ● Ponto central das rotas do frontend: frontend/src/App.jsx.
Onde se encontram os testes?	Testes do backend: backend/tests/test_main.py. Testes do frontend: não possui.
Como o software é executado?	Backend: 1. Entrar em backend/. 2. Criar e ativar um ambiente virtual. (env) 3. Executar pip install -r requirements.txt. 4. Iniciar com uvicorn app.main:app --reload. 5. API disponível em http://127.0.0.1:8000/docs. Frontend: 1. Entrar em frontend/. 2. Executar npm install. 3. Iniciar com npm start. 4. Aplicação disponível em http://localhost:3000.
Que tipos de mudanças podem ser observadas no histórico do projeto?	● Implementação e aprimoramento das funcionalidades de listagem de alimentos. ● Alterações nas telas de login, cadastro, perfil e página inicial. ● Melhorias na interface, no layout e nos estilos visuais. ● Adição de recursos de comunicação por e-mail e WhatsApp. ● Criação e execução de testes de integração. ● Atualizações e complementações da documentação no README.md. ● Integração de alterações por meio de branches e pull requests.

2.1 BASELINE

Informação	**Resultado**
Commit da baseline	9e66802ed756f794ccb236f3ea9e5c03fd85f755
Link do nosso repositório	https://github.com/DiegoBloemer/food-rescue

Build	Falha, foi necessário instalar as dependências “pydantic[email]” e react-icons não especificados nos arquivos de dependências, após instalação o projeto buildou;
Quantidade de testes	5
Testes bem sucedidos	4
Execução da aplicação	Sucesso, porém algumas funcionalidades estavam com falhas.

3 PROBLEMAS E REFATORAÇÃO

3.1 FALHAS DE MANUTENIBILIDADE IDENTIFICADOS

3.1.1 Método *datetime.utcnow()* Descontinuado

Elemento	Descrição
Problema identificado	Deprecated API - No arquivo de dependências do backend do projeto não foram definidas as versões das dependências utilizadas no arquivo *requirements.txt*, logo ao realizar a instalação são buscadas as versões mais recentes o que ocasionou erros na utilização do método descontinuado *datetime.utcnow()* da biblioteca *sqlmodel*.
Localização	Classes: *Claim* - Arquivo: *models.py;* Função: *create_access_token* - Arquivo: *auth.py;*
Evidência	Teste de *Claim* falhando e sugestão da própria biblioteca para alteração.
Consequência	Como o método foi descontinuado a sua utilização está falhando e travando os testes e inutilizando a funcionalidade de criação de claims.
Possível tratamento	Substitute Algorithm - Seria necessário substituir o método descontinuado pelo novo recomendado pela biblioteca. Além disso, seria recomendado fixar a versão de todas as dependências no arquivo *requirements.txt.*
Commit	fix: atualiza dependencias do backend

3.1.2 Navbar não é Atualizada após o Registro/Login

Elemento	Descrição
Problema identificado	Inconsistent State - Após a realização do login/cadastro a navbar não é atualizada para mostrar os novos botões profile e create.
Localização	Componente: frontend/src/components/Navbar.jsx
Evidência	O componente lê *getRole()* apenas uma vez, no *useEffect* executado na montagem. Login e cadastro alteram o *localStorage*, mas não atualizam o estado da *navbar*.
Consequência	Os menus *Profile* e *Create* não aparecem imediatamente após a autenticação, causando comportamento inconsistente na interface.
Possível tratamento	Introduce Observer - emitir um evento de autenticação após saveAuth/clearAuth
Commit	● refactor: atualiza estado de autenticacao e navbar ● fix: profile creation, navbar buttons, and updating dependency files

3.1.3 Dados Enviados Incorretamente para API

Elemento	Descrição
Problema identificado	Inconsistent API Contract
Localização	Componente: frontend/src/pages/Profile.jsx - função createProfile
Evidência	O frontend enviava email, mas o backend exigia contact_email
Consequência	A API rejeita a requisição com 422 Unprocessable Content, impedindo a criação do perfil.
Possível tratamento	Rename Variable - Alinhamento do contrato da API.
Commit	● fix: corrige contrato de dados na criacao do perfil ● fix: conflito de nomes no React

3.2 PROBLEMAS DE MANUTENIBILIDADE IDENTIFICADOS

3.2.1 Método e Dependências Instanciadas e Não Utilizadas

Elemento	Descrição
Problema identificado	Diversos arquivos possuem métodos e dependências importadas que não são efetivamente utilizadas.
Localização	frontend/src/pages/Home.jsx, linha 5: importação de authFetch, que não é utilizada. frontend/src/pages/CreateListing.jsx, linha 3: importação de FaTimes e FaEye, que não são utilizados no componente.
Evidência	[{ "resource": "/c:/Users/diego/Desktop/Cursos/UFSC/TIC/Engenharia de Software II/Trabalhos/Trabalho de Manutenção de Software/food-rescue/frontend/src/pages/Home.jsx", "owner": "sonarlint", "code": "javascript:S1128", "severity": 4, "message": "Remove this unused import of 'authFetch'.", "source": "sonarqube", "startLineNumber": 5, "startColumn": 19, "endLineNumber": 5, "endColumn": 28, "modelVersionId": 1, "origin": "extHost1" }] [{ "resource": "/c:/Users/diego/Desktop/Cursos/UFSC/TIC/Engenharia de Software II/Trabalhos/Trabalho de Manutenção de Software/food-rescue/frontend/src/pages/CreateListing.jsx", "owner": "sonarlint", "code": "javascript:S1128", "severity": 4,

	"message": "Remove this unused import of 'FaTimes', 'FaEye' .", "source": "sonarqube", "startLineNumber": 3, "startColumn": 27, "endLineNumber": 3, "endColumn": 34, "modelVersionId": 1, "origin": "extHost1" }]
Consequência	Maior tempo de entendimento e leitura do código, além de dificultar a busca pelos métodos e dependências utilizados e importados.
Possível tratamento	Remove Unused Imports e Remove Dead Dependency.

3.2.2 God Component / Mistura de Responsabilidades no Front-end

Elemento	Descrição
Problema identificado	O componente concentra estado do formulário, validação, chamadas HTTP, edição, exclusão, carregamento da lista, mensagens, transformação de datas e configuração de status.
Localização	frontend/src/pages/CreateListing.jsx, componente CreateListing, aproximadamente linhas 5–151

Evidência	`export default function CreateListing() {`
	`  const [formData, setFormData] = useState({`
	`  title: '',`
	`  description: '',`
	`  quantity: 1,`
	`  unit: 'kg',`
	`  expiresAt: '',`
	`  city: '',`
	`  address: ''`
	`  });`
	`  const [msg, setMsg] = useState('');`
	`  const [msgType, setMsgType] = useState('');`
	`  const [loading, setLoading] = useState(false);`

```
      const    [myListings,    setMyListings]    =
useState([]);
       const    [activeTab,    setActiveTab]    =
useState('create');
   const  [editingListing,  setEditingListing]  =
useState(null);

  useEffect(() => {
    if (activeTab === 'manage') {
      fetchMyListings();
    }
  }, [activeTab]);

  const fetchMyListings = async () => {
    try {
                        const    data    =    await
authFetch('/listings/my');
      setMyListings(Array.isArray(data) ? data :
[]);
    } catch (error) {
       console.error('Error fetching listings:',
error);
      setMyListings([]);
    }
  };

  const createListing = async (e) => {
    e.preventDefault();
    setLoading(true);
    setMsg('');

                          const      donorId      =
localStorage.getItem('donorId');
    if (!donorId) {
       setMsg('Please create your Donor profile
first (Profile page).');
      setMsgType('error');
      setLoading(false);
      return;
```

```
    }

    if (!formData.title || !formData.quantity ||
!formData.unit ||
        !formData.expiresAt || !formData.city) {
          setMsg('Please fill in all required
fields.');
      setMsgType('error');
      setLoading(false);
      return;
    }

    try {
      const url = editingListing
        ? `/listings/${editingListing.id}`
        : `/listings?donor_id=${donorId}`;

        const method = editingListing ? 'PUT' :
'POST';

      await authFetch(url, {
        method,
        body: JSON.stringify({
          ...formData,
          quantity: Number(formData.quantity),
                                  expires_at:   new
Date(formData.expiresAt).toISOString()
        })
      });

      setMsg(editingListing
        ? 'Listing updated successfully!'
        : 'Listing created successfully!');
      setMsgType('success');
      setEditingListing(null);
      fetchMyListings();
    } catch (error) {
      setMsg(error.message || 'Failed to process
your request');
```

```
      setMsgType('error');
    } finally {
      setLoading(false);
    }
  };

  const editListing = (listing) => {
    setFormData({
      title: listing.title,
      description: listing.description || '',
      quantity: listing.quantity,
      unit: listing.unit,
          expiresAt:  listing.expires_at.slice(0,
16),
      city: listing.city,
      address: listing.address || ''
    });
    setEditingListing(listing);
    setActiveTab('create');
  };

  const deleteListing = async (listingId) => {
     if (!window.confirm('Are you sure you want
to delete this listing?')) {
      return;
    }

    try {
       await authFetch(`/listings/${listingId}`,
{
        method: 'DELETE'
      });

      setMsg('Listing deleted successfully!');
      setMsgType('success');
      fetchMyListings();
    } catch (error) {
      setMsg('Failed to delete listing');
      setMsgType('error');
```

	<pre> } }; }</pre>
Consequência	O componente conhece detalhes de persistência, transformação e payload, concentrando funções e consequentemente tendo uma baixa coesão. Necessário separar as responsabilidades.
Possível tratamento	Extract Function: Separação entre estado/aplicação e apresentação.

3.2.3 Fat Route / Regra de Negócio e Persistência Misturadas na Rota

Elemento	Descrição
Problema identificado	Regra de Negócio e Persistência Misturadas na Rota
Localização	backend/app/routers/claims.py, função create_claim, linhas 9–28.

Evidência	<pre>create_claim(payload: ClaimCreate, session: Session = Depends(get_session)):
	    listing = session.get(Listing, payload.listing_id)
	    if not listing:
	        raise HTTPException(status_code=404, detail="Listing not found")
	    if listing.status != "OPEN":
	        raise HTTPException(status_code=400, detail="Listing is not open for claims")
	    ngo = session.get(NGO, payload.ngo_id)
	    if not ngo:
	        raise HTTPException(status_code=404, detail="NGO not found")
	    existing = session.exec(select(Claim).where(Claim.listing_id == listing.id, Claim.ngo_id == ngo.id)).first()
	    if existing:
	        raise HTTPException(status_code=409, detail="Duplicate claim")</pre>

	`claim = Claim(**payload.model_dump())`
	`session.add(claim)`
	`listing.status = "CLAIMED"`
	`session.add(listing)`
	`session.commit()`
	`session.refresh(claim)`
	`return claim`

Consequência	Método acumulando funcionalidades afetando a coesão e aumentando o acoplamento.
Possível tratamento	Extract Class: separar entre um arquivo de rota *claims.py* e um service para armazenar os dados no banco *claim_service.py.*

3.3 REFATORAÇÃO REALIZADA

Elemento	Descrição
Arquivo	backend/app/routers/claims.py
Classe / Método	Função: create_claim.
Commit	refactor(claims): extract claim creation logic from route

Antes	`create_claim(payload: ClaimCreate, session: Session = Depends(get_session)):`
	`listing = session.get(Listing, payload.listing_id)`
	`if not listing:`
	`raise HTTPException(status_code=404, detail="Listing not found")`
	`if listing.status != "OPEN":`
	`raise HTTPException(status_code=400, detail="Listing is not open for claims")`
	`ngo = session.get(NGO, payload.ngo_id)`
	`if not ngo:`
	`raise HTTPException(status_code=404, detail="NGO not found")`
	`existing = session.exec(select(Claim).where(Claim.listing_id == listing.id, Claim.ngo_id == ngo.id)).first()`

<table>
<tr><td></td><td>

```
    if existing:
                raise  HTTPException(status_code=409,
detail="Duplicate claim")
    claim = Claim(**payload.model_dump())
    session.add(claim)
    listing.status = "CLAIMED"
    session.add(listing)
    session.commit()
    session.refresh(claim)
    return claim
```

</td></tr>
<tr><td>Depois</td><td>

claims.py:

```
from fastapi import APIRouter, Depends
from sqlmodel import Session

from ..db import get_session
from ..schemas import ClaimCreate, ClaimRead
from ..services.claim_service import ClaimService

router = APIRouter(prefix="/claims", tags=["claims"])


def get_claim_service(
    session: Session = Depends(get_session),
) -> ClaimService:
    return ClaimService(session)


@router.post("", response_model=ClaimRead)
def create_claim(
    payload: ClaimCreate,
    service: ClaimService =
Depends(get_claim_service),
):
    return service.create_claim(payload)
```

claim.service():

```
from fastapi import HTTPException
from sqlmodel import Session, select
```

</td></tr>
</table>

```
from ..models import Claim, Listing, NGO
from ..schemas import ClaimCreate


class ClaimService:
    def __init__(self, session: Session):
        self.session = session

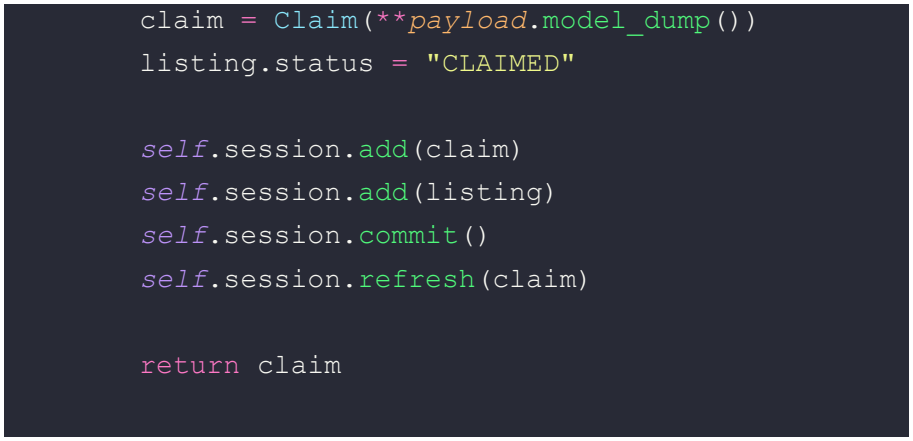
    def create_claim(self, payload: ClaimCreate) ->
Claim:
        listing = self.session.get(Listing,
payload.listing_id)
        if listing is None:
            raise HTTPException(status_code=404,
detail="Listing not found")

        if listing.status != "OPEN":
            raise HTTPException(
                status_code=400,
                detail="Listing is not open for
claims"
            )

        ngo = self.session.get(NGO, payload.ngo_id)
        if ngo is None:
            raise HTTPException(status_code=404,
detail="NGO not found")

        duplicate = self.session.exec(
            select(Claim).where(
                Claim.listing_id ==
payload.listing_id,
                Claim.ngo_id == payload.ngo_id
            )
        ).first()

        if duplicate is not None:
            raise HTTPException(status_code=409,
detail="Duplicate claim")
```

```
        claim = Claim(**payload.model_dump())
        listing.status = "CLAIMED"

        self.session.add(claim)
        self.session.add(listing)
        self.session.commit()
        self.session.refresh(claim)

        return claim
```

4 ANÁLISE, SOLICITAÇÃO E IMPLEMENTAÇÃO DE MANUTENÇÃO

4.1 SOLICITAÇÃO DE MANUTENÇÃO

Elemento	**Descrição**
Solicitação	Criar uma listagem histórica das *listings* no perfil do usuário com papel *donor*.
Descrição da Necessidade	Atualmente, o donor deveria conseguir criar e gerenciar listagens na tela de criação, mas não possui uma visualização consolidada das doações criadas diretamente no seu perfil.
Comportamento Esperado	Ao acessar o perfil de um *donor* com perfil já criado, o sistema deve buscar e exibir somente as *listings* vinculadas ao seu *donor_id*.
Regras e Restrições	• A funcionalidade deve aparecer somente para usuários com papel *donor*. • O histórico deve ser filtrado pelo *donor_id* do perfil. • Listings de outros *donors* não podem ser exibidas. • As listings devem ser ordenadas das mais recentes para as mais antigas. • O estado de carregamento, erro e lista vazia deve ser tratado na interface. • Um *donor* inexistente deve retornar 404 Not Found.
Critérios de Aceitação	• O endpoint GET /listings/donor/{*donor_id*} retorna somente listings do *donor* informado. • O endpoint retorna 404 quando o *donor* não existe. • O perfil do *donor* exibe título, quantidade,

	cidade, status e data de expiração. ● Usuários *NGO* não visualizam a seção de histórico do *donor*. ● Uma mensagem apropriada é exibida quando não existem listings. ● Os testes de back-end continuam passando. ● O build do front-end é concluído com sucesso.

4.2 ANÁLISE DE IMPACTO

Tipo de Elemento	**Elementos**
Classes	Modelo *Listing* e modelo *Donor*, utilizados na consulta relacional. Não foi necessário alterar a estrutura dos modelos.
Métodos	Novo método *list_donor_listings* em *backend/app/routers/listings.py*. Novo efeito de carregamento no componente *Profile* em *frontend/src/pages/Profile.jsx*.
Módulos	Router de listings do FastAPI, componente de perfil do React e testes de integração.
Dados	Nenhuma alteração de *schema* ou migração foi necessária. A relação existente *Listing.donor_id* já suporta a funcionalidade.
Interfaces	Novo endpoint REST GET /listings/donor/{donor_id} e nova seção visual “My Listings” no perfil do donor.
Testes	Novo teste *test_list_listings_by_donor* em *backend/tests/test_main.py*.
Dependências	Reutilização de *Session*, *select*, *Listing*, *Donor*, *ListingRead* e *authFetch*. Não foram adicionadas novas dependências de produção.
Risco de Regressão	Baixo. A alteração adiciona uma nova rota e uma seção condicional, sem modificar as rotas existentes de criação, busca ou listagem geral.

4.3 DESCRIÇÃO DA IMPLEMENTAÇÃO

A implementação da funcionalidade de histórico de anúncios no perfil do doador foi estruturada em ambas as camadas da aplicação, mantendo o padrão arquitetural desacoplado do projeto.

4.3.1 BACK-END

Foi criado o endpoint: GET /listings/donor/{donor_id}. O mesmo possui a seguinte implementação:

```
@router.get("/donor/{donor_id}", response_model=list[ListingRead])
def list_donor_listings(donor_id: int, session: Session = Depends(get_session)):
    donor = session.get(Donor, donor_id)
    if not donor:
        raise HTTPException(status_code=404, detail="Donor not found")
    return session.exec(
        select(Listing)
        .where(Listing.donor_id == donor_id)
        .order_by(Listing.id.desc())
    ).all()
```

Validações e funcionalidades realizadas:

- Valida a existência do donor;
- Filtra as listings por *Listing.donor_id*;
- Retorna 404 para donor inexistente;
- Ordena os resultados por id decrescente;
- Reutiliza o *schema* de resposta existente *ListingRead*.

4.3.2 FRONT-END

Adicionado nova seção no componente *frontend/src/pages/Profile.jsx,* o mesmo possui as seguintes características:

- Estado para as listings;
- Estado de carregamento;
- Estado de erro;
- Efeito para buscar o histórico;
- Seção condicional para usuários donor;
- Tabela com os dados das listings.
- Ocorre somente quando *role === 'donor' && existingId*

  Para carregar as informações é realizada a seguinte chamada:

```
const data = await authFetch(`/listings/donor/${existingId}`);
setListings(Array.isArray(data) ? data : []);
```

Os dados retornados são apresentados na interface da seguinte maneira:

```
<table className="table w-full">
  <thead>
    <tr className="text-[#555879]">
      <th>Title</th>
      <th>Quantity</th>
      <th>City</th>
      <th>Status</th>
      <th>Expires at</th>
    </tr>
  </thead>
  <tbody>
    {listings.map((listing) => (
      <tr key={listing.id}>
        <td>{listing.title}</td>
        <td>{listing.quantity} {listing.unit}</td>
        <td>{listing.city}</td>
        <td>{listing.status}</td>
        <td>{new Date(listing.expires_at).toLocaleDateString()}</td>
      </tr>
    ))}
  </tbody>
</table>
```

Commit de Referência: feat: implement listing of listings on donor profiles

4.4 TESTES E EVIDÊNCIAS DE REGRESSÃO

Realizada a criação da seguinte função de testes: *test_list_listings_by_donor()*, disponível no arquivo *test_main.py*. O teste realiza as seguintes análises e processos:

1. Cria dois donors;
2. Cria uma listing para cada donor;
3. Consulta o histórico do primeiro donor;
4. Verifica que somente a listing correspondente é retornada;
5. Confirma que o *donor_id* das respostas é o esperado.

Após adição dos novos testes foi realizada a execução do *script* de testes geral e todos os 6 passaram sem erros, foram lançados somente 2-3 avisos (*warnings*)

4.4 COMPARAÇÃO ENTRE IMPACTO PREVISTO E OBSERVADO

Aspecto	Impacto Previsto	Impacto Observado
Banco de dados	Nenhuma alteração estrutural esperada.	Confirmado. A funcionalidade utiliza Listing.donor_id existente.
API existente	As rotas atuais de criação, busca e listagem geral deveriam permanecer inalteradas.	Confirmado. Foi adicionada somente uma nova rota.
Front-end	O perfil deveria carregar o histórico somente para donors.	Confirmado. A seção é renderizada condicionalmente para role === 'donor'.
Isolamento dos dados	Listings de outros donors não deveriam ser retornadas.	Confirmado pelo teste automatizado com dois donors distintos.
Tratamento de erro	Falhas de carregamento deveriam ser apresentadas ao usuário.	Implementado com estado listingsError.
Lista vazia	Donor sem listings deveria visualizar uma mensagem informativa.	Implementado com mensagem específica para lista vazia.
Ordenação	As listings deveriam aparecer da mais recente para a mais antiga.	Implementado com order_by(Listing.id.desc()).
Regressão no back-end	Os testes existentes deveriam continuar passando.	Confirmado: 6 testes passaram.
Regressão no front-end	O projeto deveria continuar compilando.	Confirmado: build concluído com sucesso.
Dependências	Não deveriam ser adicionadas novas dependências de produção.	Confirmado. Foram reutilizadas as dependências existentes.

5 REFLEXÃO E CONCLUSÃO

O desenvolvimento do trabalho e do projeto proporcionou experiência prática com o processo real de manutenção de software. O principal desafio foi compreender a arquitetura desacoplada e as ferramentas de um projeto open source desconhecido, incompleto e com múltiplas funcionalidades falhas, exigindo a análise exploratória de arquivos de configuração, métodos de persistência e contratos de API. Durante a etapa de refatoração, compreendeu-se a distinção prática entre a manutenção corretiva e a refatoração de fato, atualizando o documento conforme o feedback do docente mas mantendo alguns dos tópicos anteriores. A implementação da manutenção perfectiva validou o conhecimento sobre injeção de funcionalidades em arquiteturas já existentes. A realização de testes de regressão automatizados confirmou a utilidade fundamental do baseline para garantir estabilidade contínua ao projeto, garantindo que as modificações e extensões fossem integradas de maneira coesa sem degradar ainda mais as outras funcionalidades do sistema.